UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias

Escuela Profesional de Ciencia de la Computación

- **Curso: Fundamentos de Programación**
- **Docente:** Américo Chulluncuy Reynoso

2026-1

Sesión 10:

Estructuras (struct) en C++

Contenido

1. ¿Por qué **struct**? :

   Definición y declaración. Inicialización y asignación. Acceso a miembros(I/O)

2. **Arreglo de Estructuras**: Almacenando múltiple de instancias. Recorrido y manipulación

3. **Anidamiento de una Estructura**: Modelado de relaciones jerárquicas

4. **Puntero a Estructura**: Acceso mediane operador -> Memoria dinámica y estructuras autoreferenciadas

5. **Paso de Estructura como Parámetro**: Por valor, por referencia y por puntero

6. **Funciones y Estructuras**

   Retorno por valor (seguro). Retorno por puntero con memoria dinámica

7. **Ejercicio integrador**

Tipos de datos definidos por el usuario

En C++, los tipos de datos se clasifican en dos grandes grupos:

- **Tipos de datos integrados**(primitivos/built-in)
  `int, float, double, char, bool`

- **Tipos de datos definidos por el usuario** (user-defined)
  `struct, class, enum, typedef/ using.`
  El **programador** necesita indicarle al compilador:
  1. **Cómo representar objetos** de ese tipo en la memoria.
  2. **Qué operaciones** se pueden realizar sobre él, ya que no están predefinidas en el lenguaje.

  Este tipo de datos **permite** al programador **crear estructuras más complejas, que pueden representar objetos del mundo real.**

1. Estructura

- Es un tipo de dato definido por el usuario que agrupa variables (miembros) **de diferentes tipos** bajo un mismo nombre.

- Definición y declaración

```
// Con estructura - Código organizado
struct Estudiante {
    string nombre;
    int edad;
    float nota;
};

Estudiante alumno1, alumno2, alumno3;
// Clara relación entre datos
```

Inicialización y asignación

```
// Forma 1: Inicialización por orden (C-style)
Estudiante alumno1 = {"Ana Lopez", 20, 18.5};

// Forma 2: Inicialización designada (C++20 en adelante)
Estudiante alumno2 = {.nombre = "Luis Perez", .edad = 21, .nota = 17.2};

// Forma 3: Inicialización miembro a miembro (post-declaración)
Estudiante alumno3;
alumno3.nombre = "Maria Gomez";
alumno3.edad = 19;
alumno3.nota = 19.0;
```

```
Estudiante origen = {"Carlos Ruiz", 22, 16.5};
Estudiante destino;

// Asignación entre estructuras del mismo tipo
 destino = origen;  // Copia todos los miembros
```

Acceso a miembros (Lectura y Escritura)

```
Estudiante alumno;

// --- ESCRITURA (asignar valores) ---
cout << "Ingrese nombre: ";
getline(cin, alumno.nombre);      // Escritura en miembro nombre

cout << "Ingrese edad: ";
cin >> alumno.edad;               // Escritura en miembro edad

cout << "Ingrese nota: ";
cin >> alumno.nota;               // Escritura en miembro nota

// --- LECTURA (mostrar valores) ---
cout << "\n--- DATOS DEL ESTUDIANTE ---" << endl;
cout << "Nombre: " << alumno.nombre << endl;   // Lectura de nombre
cout << "Edad: " << alumno.edad << endl;       // Lectura de edad
cout << "Nota: " << alumno.nota << endl;       // Lectura de nota
```

2. Arreglo de Estructuras

permite almacenar múltiples instancias de una estructura en posiciones contigua de memoria. Cada elemento del arreglo es una estructura completa

```
struct Contacto {
    string nombre;
    string telefono;
    string email;
};

Contacto agenda[100];  // Arreglo de 100 contactos

// Acceso al i-ésimo contacto
agenda[i].nombre = "Ana";
agenda[i].telefono = "987654321";
```

3. Anidamiento de estructuras

Ocurre cuando una estructura contiene como miembro a otra estructura. Esto permite modelar relaciones jerárquicas y composición de objetos

```
struct Coordenada {
   double latitud;
   double longitud;
};


struct Ciudad {
    string nombre;
    Coordenada ubicacion;  // Anidamiento por composición
    float temperatura;
    int humedad;
};


Ciudad lima = {"Lima", {-12.0464, -77.0428}, 22.5, 82};
cout << lima.nombre << " en latitud: " << lima.ubicacion.latitud;
```

4. Puntero a estructura

- Es una variable que almacena la dirección de memoria de una instancia de estructura. Permite acceso indirecto a los miembros mediante el operador -> (flecha) o desreferenciación con (*puntero).miembro

  Permiten el manejo dinámico de estructuras (crear modificar liberar en tiempo de ejecución)

```
Estudiante* e = new Estudiante;

    e->nombre = "Luis";
    e->edad = 21;
    e->nota = 18.2;

    cout << e->nombre;
    delete e;
```

Estructuras autoreferenciadas

- Es una estructura que contiene un puntero a su mismo tipo.
- Importancia: permite encadenar nodos y crear estructuras dinámicas
- Aplicaciones: Lista enlazada simple, pilas, colas, árboles, grafos, etc

```
struct Nodo {
    int dato;
    Nodo* siguiente; // Auto-referencia (puntero al mismo tipo)
};

// Creación dinámica
Nodo* primero = new Nodo;
primero->dato = 10;
primero->siguiente = nullptr;
```

5. Paso de Estructura como parámetro

- Las estructuras se pueden pasar a una función y devolverlas de la misma forma que cualquier otro tipo de variable: por valor (copia), por referencia (alias) o por puntero (dirección).

Tabla comparativa:

Mecanismo	Sintaxis	Modifica original	Rendimiento	Memoria
Por valor	void f(struct s)	No	Lento (copia)	Stack
Por referencia	void f(struct &s)	Sí	Rápido	Stack
Por puntero	void f(struct *s)	Sí	Rápido	Heap/Stack

```
struct Empleado {
    string nombre;
    float salario;
};

void aumentarSalarioValor(Empleado e) {  // 1. Por valor
    e.salario *= 1.10;  // Solo modifica copia local
}

void aumentarSalarioReferencia(Empleado &e) { // 2. Por referencia
    e.salario *= 1.10;  // Modifica el original
}

void aumentarSalarioPuntero(Empleado *e) { // 3. Por puntero - Modifica original
    if(e != nullptr) {
        e->salario *= 1.10;
    }
}
```

6. Funciones que retornan estructuras

- Una función puede **retornar una estructura** por valor, referencia o puntero. El retorno por valor es el más seguro, mientras que retornar referencia o puntero a variables locales produce comportamiento indefinido.

```
struct Estudiante {
    string nombre;
    int codigo;
};

Estudiante crearEstudiante() {
    Estudiante e;
    cout << "Ingrese nombre: ";
    getline(cin, e.nombre);
    cout << "Ingrese código: ";
    cin >> e.codigo;
    return e;  // Retorna copia (seguro)
}

int main() {
    Estudiante alumno = crearEstudiante();
}
```

```
Estudiante& funcionPeligrosa() {
    Estudiante e;  // Variable local
    return e;      // ¡ERROR! e se
                      destruye al salir
}

Estudiante* crearEstudianteDinamico() {
    Estudiante* e = new Estudiante;
    cout << "Ingrese nombre: ";
    getline(cin, e->nombre);
    return e;  // Válido - memoria en heap
}


int main() {
    Estudiante* alumno =
crearEstudianteDinamico();
    delete alumno;  // ¡No olvidar liberar!
}
```

Ejercicio integrador – Sistema de pedidos

Diseñe las estructuras necesarias para un sistema de delivery. Debe considerar: Restaurante, Producto, Pedido y Repartidor. Identifique relaciones de composición (anidamiento) y use arreglos donde corresponda.

```
struct Producto {
    int id;
    string nombre;
    double precio;
    string categoria;
};
```

```
struct Restaurante {
    string nombre;
    string direccion;
    string telefono;
    Producto menu[50];  // Arreglo de productos
    int totalProductos;
};
```

```
struct Repartidor {
    string nombre;
    string vehiculo;
    double latitud;
    double longitud;
    bool disponible;
};
```

```
struct Pedido {
    int numero;
    string cliente;
    Producto items[20];    // Productos del pedido
    int cantidadItems;
    double total;
    string estado; // "pendiente", "en camino", "entregado"
    Repartidor* asignado;  // Puntero al repartidor (puede cambiar)
    Restaurante* origen;   // Puntero al restaurante
};
```